Pipe support for user-defined range adaptors

Document #:	P2387R1
Date:	2021-08-09
Project:	Programming Language C++
Audience:	LEWG

Reply-to: Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

Since [P2387R0], added a feature test macro.

§ 2 Introduction

When we presented the C++23 Ranges Plan [P2214R0] to LEWG, one of the arguments we made was that the top priority item on the list was the ability to eagerly collect a range into a type (`ranges::to` [P1206R3]). During the telecon discussing that paper, Walter Brown made an excellent observation: if we gave users the tools to write their own range adaptors that would properly inter-operate with standard library adaptors (as well as other users' adaptors), then it becomes less important to provide more adaptors in the standard library.

The goal of this paper is provide that functionality: provide a standard customization mechanism for range adaptors, so that everybody can write their own adaptors.

3 Implementation Experience

To start with, there have been several implementations of the range adaptor design, that are all a little bit different. It is worth going through them all to compare how they solved the problem.

3.1 NanoRange

In Tristan Brindle's [NanoRange], we have the following approach. NanoRange uses the acronyms `raco` for **R**ange **A**daptor **C**losure **O**bject and `rao` for **R**ange **A**daptor **O**bject (both defined in 24.7.2 [range.adaptor.object]).

```
namespace nano::detail {
    // variable template to identify a Range Adaptor Closure Object
    template <typename>
    inline constexpr bool is_raco = false;

    // support for R | C to evaluate as C(R)
    template <viewable_range R, typename C>
        requires is_raco<remove_cvref_t<C>>
              && invocable<C, R>
    constexpr auto operator|(R&& lhs, C&& rhs) -> decltype(auto) {
        return FWD(rhs)(FWD(lhs));
    }

    // a type to handle merging two Range Adaptor Closure Objects together
    template <typename LHS, typename RHS>
    struct raco_pipe {
        LHS lhs;
        RHS rhs;

        // ...

        template <viewable_range R>
            requires invocable<LHS&, R>
                  && invocable<RHS&, invoke_result_t<LHS&, R>>
        constexpr auto operator()(R&& r) const {
            return rhs(lhs(FWD(r)));
        }

        // ...
    };

    // ... which is itself a Range Adaptor Closure Object
    template <typename LHS, typename RHS>
    inline constexpr bool is_raco<raco_pipe<LHS, RHS>> = true;

    // support for C | D to produce a new Range Adaptor Closure Object
    // so that (R | C) | D and R | (C | D) can be equivalent
    template <typename LHS, typename RHS>
```

```
        requires is_raco<remove_cvref_t<LHS>>
              && is_raco<remove_cvref_t<RHS>>
    constexpr auto operator|(LHS&&, RHS&&) {
        return raco_pipe<decay_t<LHS>, decay_t<RHS>>(FWD(lhs), FWD(rhs));
    }

    // ... and a convenience type for creating range adaptor objects
    template <typename F>
    struct rao_proxy : F {
        constexpr explicit rao_proxy(F&& f) : F(std::move(f)) { }
    };

    template <typename F>
    inline constexpr bool is_raco<rao_proxy<F>> = true;
}
```

And with that out of the way, NanoRange can create range adaptors fairly easily whether or not the range adaptor does not take any extra arguments (as in `join`) or does (as in `transform`). Note that `join_view_fn` *must* be in the `nano::detail` namespace in order for `rng | nano::views::join` to find the appropriate `operator|` (but `transform_view_fn` does not, since with `rng | nano::views::transform(f)` the invocation of `transform(f)` returns a `rao_proxy` which is itself a type in the `nano::detail` namespace):

join

```
namespace nano::detail {
  struct join_view_fn {
    template <viewable_range R>
        requires /* ... */
    constexpr auto operator(R&& r)
        -> join_view<all_t<R>>;
  };

  template <>
  inline constexpr
        is_raco<join_view_fn> =
        true;
}

namespace nano::views {
  // for user consumption
  inline constexpr
        detail::join_view_fn
        join{};
}
```

transform

```
namespace nano::detail {
  struct transform_view_fn_base {
    // the overload that has all the
        information
    template <viewable_range E, typename
        F>
        requires /* ... */
    constexpr auto operator()(E&& e, F&&
        f) const
        -> transform_view<all_t<E>,
        decay_t<F>>;

    // the partial overload
    template <typename F>
    constexpr auto operator()(F f) const {
        return rao_proxy{
            [f=move(f)](viewable_range
        auto&& r)
                requires /* ... */
            {
                return /* ... */;
            }};
    }
  };
}

namespace nano::views {
  // for user consumption
```

join	transform
	inline constexpr detail::transform_view_fn transform{}; }

Although practically speaking, users will not just copy again the constraints for `struct meow_view_fn` that they had written for `struct meow_view`. Indeed, NanoRange does not do this. Instead, it uses the trailing-return-type based SFINAE to use the underlying range adaptor's constraints. So `join_view_fn` actually looks like this (and it's up to `join_view` to express its constraints properly:

```
struct join_view_fn {
    template <typename E>
    constexpr auto operator()(E&& e) const
        -> decltype(join_view{FWD(e)})
    {
        return join_view{FWD(e)};
    }
};
```

§ 3.2 range-v3

In [range-v3], the approach is a bit more involved but still has the same kind of structure. Here, we have three types: `view_closure<F>` inherits from `view_closure_base` inherits from `view_closure_base_` (the latter of which is an empty class).

`view_closure<F>` is a lot like NanoRange's `rao_proxy<F>`, just with an extra base class:

```
template <typename ViewFn>
struct view_closure : view_closure_base, ViewFn
{
    view_closure() = default;

    constexpr explicit view_closure(ViewFn fn) : ViewFn(std::move(fn)) { }
};
```

The interesting class is the intermediate `view_closure_base`, which has all the functionality:

```
namespace ranges::views {
    // this type is its own namespace for ADL inhibition
    namespace view_closure_base_ns { struct view_closure_base; }
    using view_closure_base_ns::view_closure_base;
    namespace detail { struct view_closure_base_; }

    // Piping a value into a range adaptor closure object should not yield another
        closure
    template <typename ViewFn, typename Rng>
    concept invocable_view_closure =
        invocable<ViewFn, Rng>
        && (not derived_from<invoke_result_t<ViewFn, Rng>,
        detail::view_closure_base_>);
```

```
    struct view_closure_base_ns::view_closure_base : detail::view_closure_base_ {
        // support for R | C to evaluate as C(R)
        template <viewable_range R, invocable_view_closure<R> ViewFn>
        friend constexpr auto operator|(R&& rng, view_closure<ViewFn> vw) {
            return std::move(vw)(FWD(rng));
        }

        // for diagnostic purposes, we delete the overload for R | C
        // if R is a range but not a viewable_range
        template <range R, typename ViewFn>
            requires (not viewable_range<R>)
        friend constexpr auto operator|(R&&, view_closure<ViewFn>) = delete;

        // support for C | D to produce a new Range Adaptor Closure Object
        // so that (R | C) | D and R | (C | D) can be equivalent
        template <typename ViewFn, derived_from<detail::view_closure_base_> Pipeable>
        friend constexpr auto operator|(view_closure<ViewFn> vw, Pipeable pipe) {
            // produced a new closure, E, such that E(R) == D(C(R))
            return view_closure(compose(std::move(pipe), std::move(vw)));
        }
    };
}
```

And with that, we can implement `join` and `transform` as follows:

join

```
namespace ranges::views {
  struct join_view_fn {
    template <viewable_range R>
        requires /* ... */
    constexpr auto operator(R&& r)
        -> join_view<all_t<R>>;
  };

  // for user consumption
  inline constexpr
        view_closure<join_view_fn>
        join{};
}
```

transform

```
namespace ranges::views {
  struct transform_view_fn_base {
    // the overload that has all the
        information
    template <viewable_range E, typename F>
        requires /* ... */
    constexpr auto operator()(E&& e, F&& f)
        const
        -> transform_view<all_t<E>,
        decay_t<F>>;
  };

  struct transform_view_fn
    : transform_view_fn_base
  {
    using
        transform_view_fn_base::operator();

    // the partial overload
    template <typename F>
    constexpr auto operator()(F f) const {
      return view_closure(bind_back(
        transform_view_fn_base{},
        std::move(f)));
    }
  };

  // for user consumption
```

join	transform
	inline constexpr transform_view_fn transform{}; }

Compared to NanoRange, this looks very similar. We have to manually write both overloads for `transform`, where the partial overload returns some kind of special library closure object (`view_closure` vs `rao_proxy`). The primary difference here is that with NanoRange, `join_view_fn` needed to be defined in the `nano::detail` namespace and then the variable template `is_raco` needed to be specialized to `true`, while in range-v3, `join_view_fn` can actually be in any namespace as long as the `join` object itself has type `view_closure<join_view_fn>`.

3.3 gcc 10

The implementation of pipe support in [gcc-10] is quite different from either NanoRange or range-v3. There, we had two class templates: `__adaptor::_RangeAdaptorClosure<F>` and `__adaptor::_RangeAdaptor<F>`, which represent range adaptor closure objects and range adaptors, respectively.

The latter either invokes `F` if possible (to handle the `adaptor(range, args...)` case) or, if not, returns a `_RangeAdaptorClosure` specialization (to handle the `adaptor(args...)` case). The following implementation is reduced a bit, to simply convey how it works (and to use non-uglified names):

```
template <typename Callable>
struct _RangeAdaptor {
    [[no_unique_address]] Callable callable;

    template <typename... Args>
        requires (sizeof...(Args) >= 1)
    constexpr auto operator()(Args&&... args) const {
        if constexpr (invocable<Callable, Args...>) {
            // The adaptor(range, args...) case
            return callable(FWD(args)...);
        } else {
            // The adaptor(args...)(range) case
            return _RangeAdaptorClosure(
                [...args=FWD(args), callable]<typename R>(R&& r){
                    return callable(FWD(r), args...);
                });
        }
    }
};
```

The former provides piping support:

```
template <typename Callable>
struct _RangeAdaptorClosure : _RangeAdaptor<Callable>
{
    // support for C(R)
    template <viewable_range R> requires invocable<Callable, R>
    constexpr auto operator()(R&& r) const {
```

```
        return callable(FWD(r));
    }

    // support for R | C to evaluate as C(R)
    template <viewable_range R> requires invocable<Callable, R>
    friend constexpr auto operator|(R&& r, _RangeAdaptorClosure const& o) {
        return o.callable(FWD(r));
    }

    // support for C | D to produce a new Range Adaptor Closure Object
    // so that (R | C) | D and R | (C | D) can be equivalent
    template <typename T>
    friend constexpr auto operator|(_RangeAdaptorClosure<T> const& lhs,
        _RangeAdaptorClosure const& rhs) {
        return _RangeAdaptorClosure([lhs, rhs]<typename R>(R&& r){
            return FWD(r) | lhs | rhs;
        });
    }
};
```

And with that, we can implement `join` and `transform` as follows:

join

```
namespace std::ranges::views {
  // for user consumption
  inline constexpr
        __adaptor::_RangeAdaptorClosure
        join
    = []<viewable_range R> requires /* ...
        */
      (R&& r) {
        return join_view(FWD(r));
      };
}
```

transform

```
namespace std::ranges::views {
  // for user consumption
  inline constexpr
        __adaptor::_RangeAdaptor
        transform
    = []<viewable_range R, typename
        F>
        requires /* ... */
      (R&& r, F&& f){
        return
        transform_view(FWD(r),
        FWD(f));
      };
}
```

Compared to either NanoRange or range-v3, this implementation strategy has the significant advantage that we don't have to write both overloads of `transform` manually: we just write a single lambda and use class template argument deduction to wrap its type in the right facility (`_RangeAdaptorClosure` for `join` and `_RangeAdaptor` for `transform`) to provide `|` support.

This becomes clearer if we look at gcc 10's implementation of `views::transform` vs range-v3's directly:

range-v3

```
namespace ranges::views {
  struct transform_view_fn_base {
    // the overload that has all the
        information
    template <viewable_range E, typename F>
```

gcc 10

```
namespace std::ranges::views {
  // for user consumption
  inline constexpr
        __adaptor::_RangeAdaptor
        transform
```

range-v3	gcc 10

`requires /* ... */`
`constexpr auto operator()(E&& e, F&& f)`
`const`
`-> transform_view<all_t<E>,`
`decay_t<F>>;`
`};`
 	
`struct transform_view_fn`
`: transform_view_fn_base`
`{`
`using`
`transform_view_fn_base::operator();`
 	
`// the partial overload`
`template <typename F>`
`constexpr auto operator()(F f) const {`
`return view_closure(bind_back(`
`transform_view_fn_base(),`
`std::move(f)));`
`}`
`};`
 	
`// for user consumption`
`inline constexpr transform_view_fn`
`transform{};`
`}`
	`= []<viewable_range R,`
	`typename F>`
	`requires /* ... */`
	`(R&& r, F&& f){`
	`return`
	`transform_view(FWD(r),`
	`FWD(f));`
	`};`
	`}`

§ 3.4 gcc 11

The implementation of pipe support in [gcc-11] is closer to the range-v3/NanoRange implementations than the gcc 10 one.

In this implementation, `_RangeAdaptorClosure` is an empty type that is the base class of every range adaptor closure, equivalent to range-v3's `view_closure_base`:

```
struct _RangeAdaptorClosure {
    // support for R | C to evaluate as C(R)
    template <typename Range, typename Self>
        requires derived_from<remove_cvref_t<Self>, _RangeAdaptorClosure>
              && invocable<Self, Range>
    friend constexpr auto operator|(Range&& r, Self&& self) {
        return FWD(self)(FWD(r));
    }

    // support for C | D to produce a new Range Adaptor Closure Object
    // so that (R | C) | D and R | (C | D) can be equivalent
    template <typename Lhs, typename Rhs>
        requires derived_from<Lhs, _RangeAdaptorClosure>
              && derived_from<Rhs, _RangeAdaptorClosure>
    friend constexpr auto operator|(Lhs lhs, Rhs rhs) {
        return _Pipe<Lhs, Rhs>(std::move(lhs), std::move(rhs));
```

```
    }
};
```

`_RangeAdaptor` is a CRTP template that is a base class of every range adaptor object (not range adaptor closure):

```
template <typename Derived>
struct _RangeAdaptor {
    // provides the partial overload
    // such that adaptor(args...)(range) is equivalent to adaptor(range, args...)
    // _Partial<Adaptor, Args...> is a _RangeAdaptorClosure
    template <typename... Args>
        requires (Derived::arity > 1)
              && (sizeof...(Args) == Derived::arity - 1)
              && (constructible_from<decay_t<Args>, Args> && ...)
    constexpr auto operator()(Args&&... args) const {
        return _Partial<Derived, decay_t<Args>...>(FWD(args)...);
    }
};
```

The interesting point here is that every adaptor has to specify an `arity`, and the partial call must take all but one of those arguments. As we'll see shortly, `transform` has arity `2` and so this call operator is only viable for a single argument. As such, the library still implements every partial call, but it requires more input from the adaptor declaration itself.

The types `_Pipe<T, U>` and `_Partial<D, Args...>` are both `_RangeAdaptorClosures` that provide call operators that accept a `viewable_range` and eagerly invoke the appropriate functions (both, in the case of `_Pipe`, and a `bind_back`, in the case of `_Partial`). Both types have appeared in other implementations already.

And with that, we can implement `join` and `transform` as follows:

join

```
namespace std::ranges::views {
  struct Join :
        _RangeAdaptorClosure {
    template <viewable_range R>
        requires /* ... */
    constexpr auto operator()
        (R&& r) const
        -> join_view<all_t<R>>;
  };

  // for user consumption
  inline constexpr Join join;
}
```

transform

```
namespace std::ranges::views {
  struct Transform : _RangeAdaptor<Transform>
        {
    template <viewable_range R, typeanme F>
      requires /* ... */
    constexpr auto operator()(R&& r, F&& f)
        const
      -> transform_view<all_t<R>, F>;

    using
        _RangeAdaptor<Transform>::operator();
    static constexpr int arity = 2;
  };

  // for user consumption
  inline constexpr Transform transform;
}
```

This is longer than the gcc 10 implementation in that we need both a type and a variable, whereas before we only needed the lambda. But it's still shorter than either the NanoRange or range-v3 implementations in that we do not need to manually implement the partial overload. The library does that for us, we simply have to provide the *using-declaration* to bring in the partial `operator()` as well as declare our `arity`.

§ 3.5 msvc

The [msvc] implementation is also worth sharing as it is probably the most manual of all the implementations. While range-v3 and NanoRange require you to manually write the partial calls yourself, they both provide library wrappers that do the binding for you (`bind_back` and `rao_proxy`, respectively), in the msvc implementation, each range adaptor actually has its own private partial call implementation.

As such, the implementations of just `join` and `transform` look like (slightly reduced for paper-ware):

join

```
namespace std::ranges::views {
  struct _Join_fn :
        _Pipe::_Base<_Join_fn> {
    template <viewable_range R>
      requires /* ... */
    constexpr auto operator()(R&&
        r)
      -> join_view<all_t<R>>;
  };

  inline constexpr _Join_fn join;
}
```

transform

```
namespace std::ranges::views {
  class _Transform_fn {
    template <class F>
    struct Partial :
        _Pipe::_Base<_Partial<F>> {
      F f;

      template <viewable_range R>
      constexpr auto operator()(R&& r)
        const&
        -> decltype(transform_view(FWD(r),
        f))
      {
        return transform_view(FWD(r), f);
      }

      template <viewable_range R>
      constexpr auto operator()(R&& r) &&
        -> decltype(transform_view(FWD(r),
        move(f)))
      {
        return transform_view(FWD(r),
        move(f));
      }
    };

  public:
    // the overload that has all the
        information
    template <viewable_range R, class F>
    constexpr auto operator()(R&& r, F f)
      -> decltype(transform_view(FWD(r),
        move(f)))
    {
      return transform_view(FWD(r),
        move(f));
```

join	transform

	`  }`
	
	`  // the partial overload`
	`  template <copy_constructible F>`
	`  constexpr auto operator()(F f) {`
	`  return Partial<F>{.f=move(f)};`
	`  }`
	`};`
	
	`inline constexpr _Transform_fn transform;`
	`}`

Otherwise, the MSVC implementation of the range adaptor closure functionality (which it calls `_Pipe::_Base<D>`) is similar enough to gcc 11's implementation (which it calls `_RangeAdaptorClosure`). The primary difference is that the former is a CRTP base class template while the latter is simply a base class.

4 Problem Space

Ultimately, there are two separate problems here.

4.1 Range Adaptor Closure Objects

We need to be able to declare a *range adaptor closure object* (24.7.2 [range.adaptor.object]), which has the following requirements (where `R` is some `viewable_range`, and `C` and `D` are range adaptor closure objects):

— `C(R)` and `R | C` are equivalent

— `R | C | D` and `R | (C | D)`

It is up to the user to provide the call operator to make `C(R)` work, but it needs to be up to whatever the library design is to make `R | C` work (and end up invoking `C`) and to make `C | D` work (to produce a new range adaptor closure object).

Because the library needs to provide operator overloads, the design needs to be such that it is actually possible for those operator overloads to be discovered. The five implementations presented above have three different approaches to this:

1. The closure object type is declared in the namespace where the `operator|`s are declared (NanoRange)
2. The closure object type inherits from a regular base class (gcc 11) or CRTP base class (MSVC) which defines those `operator|`s.
3. The closure object type is a specialization of a library class template, with the actual function object type as a template parameter (range-v3, gcc 10).

Of these, the NanoRange option is a non-starter since we don't want to have everyone adding all of their types into `std::ranges`.

While a regular base class (gcc 11) is easier to use (by virtue of simply being less to type) than a CRTP base class (msvc), it has the downside of the multiple-instances-of-the-same-empty-base problem (see also [LWG3549]). In gcc 11's implementation, the `C | D` overload produces an object that looks like:

```
// A range adaptor closure that represents composition of the range
// adaptor closures _Lhs and _Rhs.
template<typename _Lhs, typename _Rhs>
  struct _Pipe : _RangeAdaptorClosure
  {
    [[no_unique_address]] _Lhs _M_lhs;
    [[no_unique_address]] _Rhs _M_rhs;
    // ...
  };
```

This object contains three copies of `_RangeAdaptorClosure`, which means it has to be at least 3 bytes wide, even if `_Lhs` and `_Rhs` are both empty.

That reduces our choice to either providing a class template that users inherit from CRTP-style (MSVC) or a class template that users use to wrap their type (gcc 10/range-v3). There isn't that much of a difference between the two as far as implementing a range adaptor closure object goes - it's just a question of where you put the library type. But there *is* a different as far as diagnostics are concerned. With the `views::join` example, it's a question of whether an error message will contain the type `JoinFn` or whether it will contain the type `std::ranges::range_adaptor_closure<JoinFn>` in it. Having the former is strictly better.

This paper proposes the MSVC approach: having a CRTP base class that implements the range adaptor closure design.

§ 4.2 Range Adaptor Objects

We need to be able to declare a *range adaptor object* (24.7.2 [range.adaptor.object]). This part is more complicated. For multi-argument range adaptors, we need the following forms to be equivalent:

- `adaptor(range, args...)`
- `adaptor(args...)(range)`
- `range | adaptor(args...)`

Where `adaptor(args...)` produces a range adaptor closure object.

While the various implementation approaches for the range adaptor closure problem were fairly similar (the library has to provide two `operator|`s, there really aren't that many ways to provide them), the implementations presented here have very different approaches to making multi-argument range adaptors more convenient - which range from extremely manual (MSVC) to effortless (gcc 10). There's such a range of implementations here that it's quite difficult to actually say which is the "right" one, or which one could be considered "standard practice." This is still an area being experimented on.

But importantly, the standard library also doesn't *need* to solve this problem. As soon as the standard library can provide a common mechanism for users to create a range adaptor closure object that can play well with others, users can implement their range adaptors any way they like. Perhaps some new, as-yet undiscovered approach comes around in a few years that is superior to the other alternatives and we can standardize that one for C++26. Perhaps a language solution (like `|>`) gets finalized and this becomes less important too.

Rather than try to introduce a mechanism like gcc 10's or gcc 11's approach, this paper actually proposes no approach. Or, put differently, this paper proposes the MSVC approach for the range adaptor object problem too.

However, one approach to the range adaptor problem (range-v3's) involves using `bind_back` to create a new range adaptor closure object. There was previously a proposal to add `bind_back` as a new function adaptor to the standard library [P0356R0], though it was removed in R1 of that paper due to lack of compelling uses cases. This problem seems compelling to me, and I'd expect some users to use `bind_back` to solve it.

This is sufficient to provide, for instance, gcc 10's solution as a small standalone library:

```
template <typename F>
class closure : public std::ranges::range_adaptor_closure<closure<F>> {
    F f;
public:
    constexpr closure(F f) : f(f) { }

    template <std::ranges::viewable_range R>
        requires std::invocable<F const&, R>
    constexpr operator()(R&& r) const {
        return f(std::forward<R>(r));
    }
};

template <typename F>
class adaptor {
    F f;
public:
    constexpr adaptor(F f) : f(f) { }

    template <typename... Args>
    constexpr operator()(Args&&... args) const {
        if constexpr (std::invocable<F const&, Args...>) {
            return f(std::forward<Args>(args)...);
        } else {
            return closure(std::bind_back(f, std::forward<Args>(args)...));
        }
    }
};
```

With the familiar `join` and `transform` examples showing up as:

```
inline constexpr closure join
    = []<viewable_range R> requires /* ... */
```

```
      (R&& r) {
        return join_view(FWD(r));
      };

inline constexpr adaptor transform
    = []<viewable_range R, typename F> requires /* ... */
      (R&& r, F&& f){
        return transform_view(FWD(r), FWD(f));
      };
```

5 Proposal

This paper proposes two additions to the standard library:

First, a new class template `std::ranges::range_adaptor_closure<T>` that range adaptor closure objects will have to inherit from. For existing range adaptor closure objects (like `std::views::join`) and the partially applied results from the existing range adaptor objects (like `std::views::transform(f)`), it will be up to the standard library to properly handle them (whether changing those types to inherit from this new thing or having `range_adaptor_closure` additionally recognize the particular implementation's preexisting range adaptor closure marker instead).

Second, a new function adaptor `std::bind_back`, such that `std::bind_back(f, ys...)(xs...)` is equivalent to `f(xs..., ys...)`.

5.1 Wording for `range_adaptor_closure`

Add `range_adaptor_closure` to 24.2 [ranges.syn]:

```
#include <compare>              // see [compare.syn]
#include <initializer_list>     // see [initializer.list.syn]
#include <iterator>             // see [iterator.synopsis]

namespace std::ranges {

+ // [range.adaptor.object], range adaptor objects
+ template<class D>
+   requires is_class_v<D> && same_as<D, remove_cv_t<D>>
+ class range_adaptor_closure { };

  // [view.interface], class template view_interface
  template<class D>
    requires is_class_v<D> && same_as<D, remove_cv_t<D>>
  class view_interface;
}
```

Change 24.7.2 [range.adaptor.object] [Editor's note: We have to relax these requirements because can't enforce them on future user-defined range adaptors, and we'll need to do this for `ranges::to` anyway which we'll want to call a range adaptor closure object]:

1 A *range adaptor closure object* is a unary function object that accepts a ~~`viewable_range`~~ `range` argument ~~and returns a `view`~~. For a range adaptor closure object `C` and an expression `R` such that `decltype((R))` models ~~`viewable_range`~~ `range`, the following expressions are equivalent ~~and yield a `view`~~:

```
C(R)
R | C
```

Given an additional range adaptor closure object `D`, the expression `C | D` is well-formed and produces another range adaptor closure object such that the following two expressions are equivalent:

```
R | C | D
R | (C | D)
```

? An object `t` of type `T` is a range adaptor closure object if `T` models `derived_from<range_adaptor_closure<T>>`, `T` has no other base classes of type `range_adaptor_closure<U>` for any other type `U`, and `T` does not model `range`.

? The template parameter `D` for `range_adaptor_closure` may be an incomplete type. Before any expression of type *cv* `D` appears as an operand to the `|` operator, `D` shall be complete and model `derived_from<range_adaptor_closure<D>>`. The behavior of an expression involving an object of type *cv* `D` as an operand to the `|` operator is undefined if overload resolution selects a program-defined `operator|` function.

§ 5.2 Wording for `bind_back`

Add `bind_back` to 20.14.2 [functional.syn]. The wording will go into the same section, so rename the clause from [func.bind.front] to [func.bind.partial]:

```
namespace std {

- // [func.bind.front], function template bind_front
+ // [func.bind.partial], function templates bind_front and bind_back
  template<class F, class... Args> constexpr unspecified bind_front(F&&,
        Args&&...);
+ template<class F, class... Args> constexpr unspecified bind_back(F&&,
        Args&&...);
}
```

Rename the 20.14.14 [func.bind.front] clause to [func.bind.partial] (Function templates `bind_front` and `bind_back`) and extend the wording to handle both cases:

```
  template<class F, class... Args>
    constexpr unspecified bind_front(F&& f, Args&&... args);
+ template<class F, class... Args>
+   constexpr unspecified bind_back(F&& f, Args&&... args);
```

1 Within this subclause:

(1.1) — `g` is a value of the result of a `bind_front` or `bind_back` invocation,

(1.2) — `FD` is the type `decay_t<F>`,

(1.3) — `fd` is the target object of `g` ([func.def]) of type `FD`, direct-non-list-initialized with `std::forward<F>(f)`,

(1.4) — `BoundArgs` is a pack that denotes `decay_t<Args>...`,

(1.5) — `bound_args` is a pack of bound argument entities of `g` ([func.def]) of types `BoundArgs...`, direct-non-list-initialized with `std::forward<Args>(args)...`, respectively, and

(1.6) — `call_args` is an argument pack used in a function call expression ([expr.call]) of `g`.

2 *Mandates*:

```
is_constructible_v<FD, F> &&
is_move_constructible_v<FD> &&
(is_constructible_v<BoundArgs, Args> && ...) &&
(is_move_constructible_v<BoundArgs> && ...)
```

is `true`.

3 *Preconditions*: `FD` meets the *Cpp17MoveConstructible* requirements. For each T_i in `BoundArgs`, if T_i is an object type, T_i meets the *Cpp17MoveConstructible* requirements.

4 *Returns*: A perfect forwarding call wrapper g with call pattern:

(4.1) — `invoke(fd, bound_args..., call_args...)` for a `bind_front` invocation, or

(4.2) — `invoke(fd, call_args..., bound_args...)` for a `bind_back` invocation.

5 *Throws*: Any exception thrown by the initialization of the state entities of `g` ([func.def]).

§ 5.3 Feature-test macro

Bump the value of `__cpp_lib_ranges` in 17.3.2 [version.syn]:

```
- #define __cpp_lib_ranges                    202106L
+ #define __cpp_lib_ranges                    2021XXL
    // also in <algorithm>, <functional>, <iterator>, <memory>, <ranges>
```

§ 6 References

[gcc-10] Patrick Palka. 2020. `<ranges>` in gcc 10.
https://github.com/gcc-mirror/gcc/blob/860c5caf8cbb87055c02b1e77d04f658d2c75880/libstdc%2B%2B-v3/include/std/ranges

[gcc-11] Patrick Palka. 2021. `<ranges>` in gcc 11.
https://github.com/gcc-

mirror/gcc/blob/5e0236d3b0e0d7ad98bcee36128433fa755b5558/libstdc%2B%2B-v3/include/std/ranges

[LWG3549] Tim Song. view_interface is overspecified to derive from view_base. https://wg21.link/lwg3549

[msvc] Casey Carter. 2020. `<ranges>` in msvc. https://github.com/microsoft/STL/blob/18c12ab01896e73e95a69ceba9fbd7250304f895/stl/inc/ranges

[NanoRange] Tristan Brindle. 2017. NanoRange. https://github.com/tcbrindle/nanorange

[P0356R0] Tomasz Kamiński. 2016-05-22. Simplified partial function application. https://wg21.link/p0356r0

[P1206R3] Corentin Jabot, Eric Niebler, Casey Carter. 2020-11-22. ranges::to: A function to convert any range to a container. https://wg21.link/p1206r3

[P2214R0] Barry Revzin, Conor Hoekstra, Tim Song. 2020-10-15. A Plan for C++23 Ranges. https://wg21.link/p2214r0

[P2387R0] Barry Revzin. 2021-06-12. Pipe support for user-defined range adaptors. https://wg21.link/p2387r0

[range-v3] Eric Niebler. 2013. Range library for C++14/17/20, basis for C++20's std::ranges. https://github.com/ericniebler/range-v3/